

Project - High Level Design

On

IaC Provisioning for Agriculture System

Course Name: Deveops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Pawani Shah	EN22CS301700
2.	Praniti Kshirsagar	EN22CS301726
3.	Pranjal Patidar	EN22CS301730
4.	Prafulla Kumar Verma	EN22CS301715
5.	Ritik Thakur	EN22CS301811

Group Name: Group 08D7

*Project Number:*DO-31

Industry Mentor Name: Aashruti Shah

University Mentor Name: Shyam Pathak

Academic Year: Jan 202c - May 202c

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

IaC Provisioning for Agriculture System is a DevOps project where infrastructure and applications for an agriculture support system are automatically deployed using Infrastructure as Code (IaC). Tools like Git are used for version control, Docker for containerizing the application, and Jenkins for automating the build, test, and deployment process through CI/CD pipelines

1.1 Scope of the Document

This document provides a **detailed high-level design of the Krishi AI Assistant system**, focusing on:

- Docker-based containerization
- Image creation and container lifecycle
- Deployment architecture and execution flow
- Runtime networking and service communication
- Integration with external AI services (Groq APIs)

The scope includes **actual implementation details**, such as:

- Docker images created
- Containers running in the system
- Port mappings and deployment validation

1.2 Intended Audience

- DevOps Engineers
- Backend Developers
- Deployment Engineers

- System Architects
- QA Engineers

1.3 System Overview

Krishi AI Assistant is a **containerized AI-powered agricultural system** designed to:

- Analyze crop images and farmer inputs
- Provide disease diagnosis and treatment suggestions
- Estimate market price and insurance options

Deployment Overview

The system is deployed using:

- **Frontend Container (React Application)**
- **Backend Container (FastAPI Application)**
- **External AI APIs (Groq - LLM + Vision)**

Key Deployment Characteristics

- Fully containerized architecture
- Independent services (frontend C backend)

2. System Design

2.1 Application Design

Architecture Type

- Client-Server Architecture
- Containerized deployment model

Deployment Architecture

User Browser
↓
Frontend Container (React - Port 3000)
↓
Backend Container (FastAPI - Port 8000)
↓
External Groq APIs

Containerization Strategy

- Each service is packaged as a Docker image
- Containers are instantiated from these images
- Services are isolated but communicate via network

2.2 Process Flow :

A. Build-Time Flow

Source Code
↓
Dockerfile Execution
↓
Docker Image Creation
↓
Images Stored Locally

B. Run-Time Flow

Docker Compose Up
↓
Containers Created
↓
Docker Network Initialized
↓
Ports Exposed
↓
Application Ready

C. Request Processing Flow

1. User interacts with frontend
1. Frontend sends API request to backend
2. Backend processes request
3. Backend calls Groq AI APIs
4. Response returned to frontend
5. Output displayed to user

2.3 Information Flow

Input

- Crop image
- Crop details (location, type, issue)
- Voice/text input

Processing

- Backend API processing
- AI model interaction (Groq APIs)

Output

- Disease identification
- Suggested treatment
- Urgency level
- Market price estimation
- Insurance information

2.4 Components Design

1. Frontend Component

Docker Image:

- agriculture-support-triage-agent-frontend

Details:

- Built using React.js
- Lightweight image (~63MB)

Container Configuration:

- Running container
- Port mapping : 0.0.0.0:3000 → 80/tcp

Responsibilities:

- User interface
- Input collection
- API communication

2. Backend Component**Docker Image:**

- agriculture-support-triage-agent-backend

Details:

- Built using Python + FastAPI
- Handles AI integration

Image Size:

- ~1.03 GB

Container Configuration:

- Running container
- Port mapping: 0.0.0.0:8000 → 8000/tcp

Responsibilities:

- API handling
- Business logic

- AI processing
- Authentication

3. Supporting Component (Portainer)

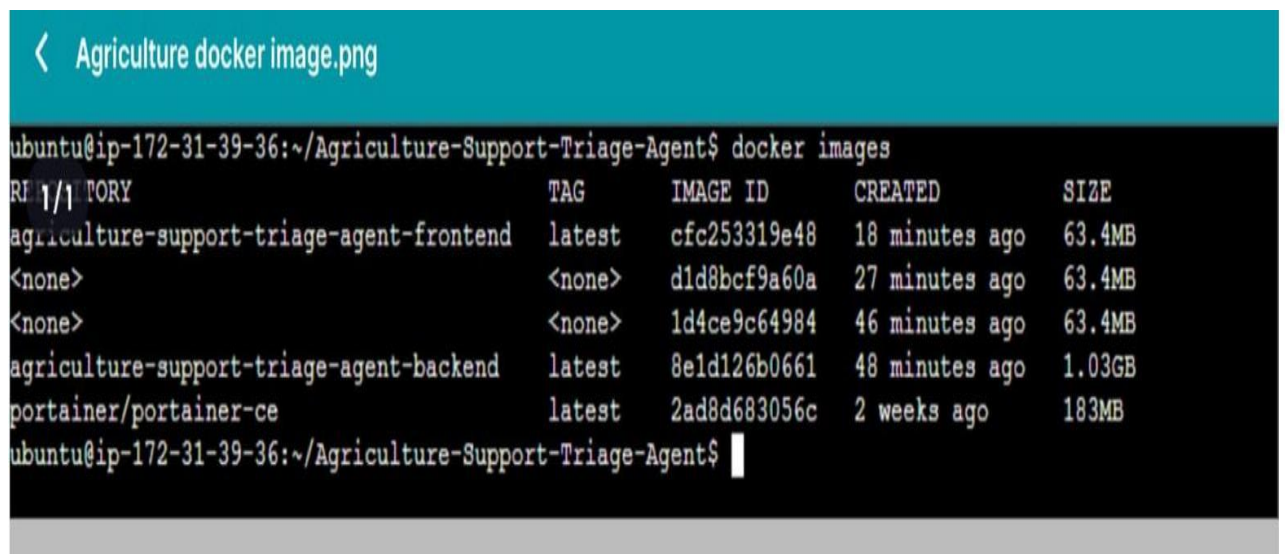
Docker Image:

- portainer/portainer-ce

Purpose:

- Container monitoring
- UI-based Docker management

4. Docker Images



```
< Agriculture docker image.png
ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$ docker images
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
agriculture-support-triage-agent-frontend  latest   cfc253319e48  18 minutes ago 63.4MB
<none>                                     <none>    d1d8bcf9a60a  27 minutes ago 63.4MB
<none>                                     <none>    1d4ce9c64984  46 minutes ago 63.4MB
agriculture-support-triage-agent-backend   latest   8e1d126b0661  48 minutes ago 1.03GB
portainer/portainer-ce                    latest   2ad8d683056c  2 weeks ago   183MB
ubuntu@ip-172-31-39-36:~/Agriculture-Support-Triage-Agent$
```

5. Docker Containers (Actual Running State)

< Agriculture docker container.png

1/1

```
shuntu@ip-172-31-36:~/Agriculture-Support-Triage-Agent$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
ee031cf4e00   portainer/portainer-ce:latest      "/portainer"           5 minutes ago  Up 5 minutes  9000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp, 0.0.0.0:8001->8000/tcp, [::]:8001->8000
/tcp   portainer
0262f5d343fc   agriculture-support-triage-agent-frontend  "/docker-entrypoint..." 17 minutes ago  Up 17 minutes  0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
agriculture-support-triage-agent-frontend-1
ch244od8169d   agriculture-support-triage-agent-backend  "uvicorn main:app --w..." 45 minutes ago  Up 45 minutes  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
agriculture-support-triage-agent-backend-1
shuntu@ip-172-31-36:~/Agriculture-Support-Triage-Agent$
```

2.5 Key Design Considerations

1. Container Isolation

- Frontend and backend run independently
- No dependency conflicts

2. Port Mapping

- Frontend → 3000
- Backend → 8000
- Ensures external accessibility

3. Environment Configuration

- Managed via .env file
- Secure API keys (Groq API)

4. Scalability

- Containers can be scaled independently
- Backend scaling for high load

5. Performance Optimization

- Lightweight frontend image

- Backend optimized using Python base image

6. Fault Tolerance

- Containers can restart automatically
- Logs available for debugging

2.6 API Catalogue

Authentication APIs

- POST /signup
- POST /login

Core APIs

- POST /analyze
- GET /market-price
- GET /insurance

System APIs

- /docs (Swagger UI for testing APIs)

3. Data Design

3.1 Data Model

The system uses a lightweight relational data model (SQLite) to store user and request-related information.

Core Entities :

1. Users Table

- user_id (Primary Key)
- username
- password

2. Requests Table

- request_id (Primary Key)
- user_id (Foreign Key)
- crop_type
- location
- image_path
- urgency_level
- response_output

Data Storage Strategy

- SQLite database stored inside backend container
- Can be mounted as Docker volume for persistence

3.2 Data Access Mechanism

- Backend uses **SQLModel / ORM layer** to interact with database
- API endpoints perform CRUD operations
- **Flow**

API Request → FastAPI Service → ORM Layer → SQLite DB

Deployment Relevance

- DB runs inside backend container
- Data accessed locally within container runtime

3.3 Data Retention Policies

- User data stored until account deletion

- Request history stored for analytics and improvement
- Logs retained for debugging and monitoring

Future Enhancement

- Add automated cleanup jobs
- Move to cloud database for long-term storage

3.4 Data Migration

Current:

- No migration required (SQLite-based local storage)

Future:

- SQLite → PostgreSQL migration
- Use migration tools like Alembic

4. Interfaces

4.1 User Interface (Frontend)

- Built using React.js
- Accessible via:

<http://3.110.128.84>

Features:

- Image upload
- Voice input
- Chat interface

- Result display/Output text

4.2 API Interface (Backend)

Accessible via:

<http://3.110.128.84:8000/health>

4.3 External Interfaces

Groq API

- Used for:
 - Vision processing
 - LLM responses

4.4 Container Interface (Deployment View)

- **Frontend Container :**
0.0.0.0:3000 → 80/tcp
- **Backend Container :**
 - 0.0.0.0:8000 → 8000/tcp
 - This enables external access to services

5. State and Session Management

State Management

- Stateless backend APIs
- Each request processed independently

Session Management

- JWT-based authentication
- Token stored on client side

Deployment Perspective

- No session stored in container
- Makes system scalable (stateless design)

6. Caching

Current Implementation :

- No caching implemented

Future Enhancements :

Redis caching layer

- **Cache:**
 - API responses
 - Market data
 - AI results

Deployment Impact :

- Redis can be added as separate container

7. Non-Functional Requirements

7.1 Security Aspects :

1. Authentication

- JWT-based login system

2. Data Security

- API keys stored in .env
- No hardcoded secrets

3. Container Security

- Isolated containers
- Limited exposure via ports

4. Network Security

- Only required ports exposed:
 - 3000 (Frontend)
 - 8000 (Backend)

7.2 Performance Aspects :

1. Container Performance

From your Docker images:

Component Size		Impact
Frontend	~63MB	Fast startup
Backend	~1.03GB	Heavy due to AI dependencies

Docker Images :

- agriculture-support-triage-agent-frontend → 63MB
- agriculture-support-triage-agent-backend → 1.03GB
- <none> → intermediate images
- portainer/portainer-ce → monitoring

Docker Containers :

- Frontend container running on port 3000
- Backend container running on port 8000
- Portainer running for monitoring

2. API Performance

- FastAPI async handling
- Efficient request processing

3. Deployment Performance

- Docker enables quick startup
- Containers restart quickly

4. Optimization Opportunities

- Use lightweight Python image
- Add caching layer
- Use CDN for frontend

8. References

- Docker Documentation
- FastAPI Documentation
- React Documentation
- Groq API Documentation